

Report - Version 4: Introduction to TensorFlow

Introduction

Version 4 rebuilds the neural character language model from Version 3 with TensorFlow.

The corpus, vocabulary, adjacent-character task, numerical representation, 27 by 27 weight matrix, training and validation split, shuffled mini-batches, cross-entropy objective, and autoregressive generator remain unchanged. TensorFlow tensors and variables replace NumPy arrays in the model calculations, while automatic differentiation replaces the manually implemented gradients.

```
Characters
↓
Integer identifiers
↓
TensorFlow one-hot tensors
↓
Trainable tf.Variable
↓
Automatic gradients
↓
Weight updates
↓
Generated text
```

Goal of Version 4

The model should be able to:

- reuse the corpus, vocabulary, numerical encoding, and neural bigram architecture from Version 3;
- represent one-hot inputs as TensorFlow tensors and trainable weights as a `tf.Variable`;
- calculate stable cross-entropy directly from logits with TensorFlow;
- calculate weight gradients automatically with `tf.GradientTape`;
- update the weights with the same mini-batch gradient descent procedure;
- preserve training and validation monitoring and reproducible text generation;
- verify that automatic gradients match the manual gradient formula from Version 3.

Architecture

The complete Version 4 flow is:

```
Corpus: 440 characters
↓
Vocabulary: 27 characters
↓
439 adjacent input-target pairs
↓
TensorFlow one-hot tensor: (439, 27)
↓
Trainable tf.Variable: (27, 27)
↓
Reproducible 80 / 20 example split
↓
GradientTape mini-batch updates
↓
Train and validation loss monitoring
↓
Autoregressive generator
```

The context still contains one character. Version 4 therefore remains the same neural character bigram model used in Version 3.

Unchanged data split and mini-batches

A local NumPy generator with seed 42 creates the same reproducible permutation of the 439 example indices. TensorFlow gather operations select the corresponding training and validation tensors.

```
training examples = 351
validation examples = 88
batch size = 32
updates per epoch = 11
```

Only the training examples are shuffled and used for parameter updates. Validation examples measure loss without modifying the model. The example-level split keeps the same intentional limitations documented in Version 3.

TensorFlow numerical model

Each input character is represented by a TensorFlow one-hot tensor of length 27. The model stores its trainable 27 by 27 weight matrix in a `tf.Variable`.

```
Input tensor shape: (439, 27)
Weight matrix shape: (27, 27)
Trainable parameters: 729
Tensor dtype: float32
```

A TensorFlow random generator initialized with seed 42 produces the initial weights. The variable keeps a fixed shape and can be updated directly during training.

Stable TensorFlow loss

TensorFlow matrix multiplication produces the logits. Sparse softmax cross-entropy then measures the prediction error directly from the logits and integer target IDs.

```
logits = tf.matmul(inputs, weights)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

The combined TensorFlow operation handles numerical stability internally. Softmax is still used separately when probabilities are needed for inspection and text generation.

Controlled execution and reproducibility

The notebook explicitly selects CPU execution because this small model does not require CUDA. TensorFlow backend logs are suppressed to keep the output clean.

Explicit seeds control TensorFlow weight initialization, NumPy data splitting, mini-batch shuffling, and text sampling. Repeated runs are deterministic within the same software environment.

Automatic differentiation

For each mini-batch, TensorFlow records the forward calculations and automatically derives the loss with respect to the trainable weight matrix:

- `tf.GradientTape` records the operations used to calculate the loss;
- `tape.gradient` calculates the weight gradient automatically;
- `assign_sub` applies the same gradient descent update used in Version 3;
- training and validation loss are measured after every epoch.

```
with tf.GradientTape() as tape:
    batch_loss = calculate_loss(weights, inputs, targets)
    weights_gradient = tape.gradient(batch_loss, weights)
    weights.assign_sub(learning_rate * weights_gradient)
```

Training configuration:

```
Learning rate: 1.0
Batch size: 32
Epochs: 100
Seed: 42
```

Training result

```
Initial train loss: 3.295990
Final train loss: 1.987506
Initial validation loss: 3.297369
Final validation loss: 2.903532
Best validation loss: 2.893411
Best validation epoch: 64
```

Training loss decreases throughout the run. Validation loss initially decreases, reaches its minimum at epoch 64, and then rises slightly because only the training loss is directly optimized.

The results remain very close to Version 3. Small differences come from the TensorFlow random generator and float32 calculations. The notebook records the best epoch for analysis but uses the final epoch-100 weights for generation.

Gradient verification

The tests compare the automatic TensorFlow gradient with the manual gradient formula introduced in Version 3.

```
automatic_gradient = tape.gradient(loss, weights)
target_vectors = tf.one_hot(targets, depth=vocabulary_size)
logits_gradient = (probabilities - target_vectors) / batch_size
manual_gradient = inputs.T @ logits_gradient
np.allclose(automatic_gradient, manual_gradient)
```

The matching values show that automatic differentiation performs the same underlying calculation that was previously implemented by hand.

Learned probabilities

After training, one row of the TensorFlow weight variable can still be interpreted as a next-character distribution for a chosen context.

For the character m, the highest probabilities are:

```
o: 0.2497 e: 0.1818 a: 0.1810
s: 0.1126 p: 0.1121 space: 0.0428
```

Neural text generation

TensorFlow calculates the probability distribution, while a local NumPy generator samples the next character. The sampled character becomes the next input, preserving the autoregressive loop from Version 3.

The demonstration generates 300 characters from the final weights. The same seed reproduces the same generated text.

Included checks

The notebook assertions verify TensorFlow tensor and variable types, one-hot and parameter shapes, data splits, finite losses, normalized probabilities, matching manual and automatic gradients, deterministic retraining, reproducible generation, and requested output length.

A complete top-to-bottom run ends with the message All checks passed.

What Version 4 teaches

The central progression is:

tensor -> variable -> loss -> tape -> gradient -> update -> verify -> sample

Version 4 connects manual numerical training with framework-based training. The predictive model remains intentionally unchanged so that the educational focus stays on TensorFlow tensors, variables, and automatic differentiation.

Interpretation

TensorFlow does not make the model more expressive. It automates differentiation and provides stable tensor operations while preserving the same character bigram architecture and gradient descent procedure.

The close agreement with Version 3, together with the explicit gradient comparison, shows that both implementations optimize the same objective.

Limitations

- the context contains one character only;
 - the corpus is small and embedded in the notebook;
 - the random example-level split can place repeated character transitions in both training and validation;
 - execution is eager and CPU-based, with no optimized input pipeline;
 - there is no early stopping, checkpoint restoration, hidden layer, or embedding;
 - validation is not a robust estimate of generalization to a different corpus;
 - generated text preserves local character patterns but has no long-term coherence.
-

Conclusion

Version 4 rebuilds the NumPy neural character model with TensorFlow while preserving its data, architecture, training procedure, and generator. TensorFlow tensors and variables represent the model, automatic differentiation replaces manual gradients, and direct gradient verification connects the two implementations.

The result prepares the project for Version 5, where learned embeddings and larger context windows can increase the model's representational capacity.